

Research Proposal – Do Planner-Executor Agents Justify Their Cost in Constraint-Heavy Travel Planning?

Marius Inselseth

University of Twente

The Netherlands

m.inselseth@student.utwente.nl

1 Introduction

Large language models (LLMs) are increasingly being used as autonomous agents that solve complex multi-step problems through tool use and iterative reasoning. Techniques such as chain-of-thought (COT) prompting [5] and ReAct [8] have established strong single-agent baselines across a range of reasoning and planning benchmarks. As AI has advanced, so has the difficulty of the tasks it's asked to perform

Building on the foundation of thinking techniques, work has shown that Multi-agent systems have shown significant performance gains over single-agent flows on collaborative reasoning tasks. [1]. Popular multi agent flows like, planners and or verifiers have become standard practice in some consumer AI's like Anthropic's Claude code offering a planner.

However, the improvements come at a cost. Multi-agent systems incur larger runtime and token cost compared against single-agent baselines due to the added context and thinking times. Multi-agent communication topologies have been shown to consume $2-11.8\times$ [9] more tokens than single-agent baselines on standard reasoning benchmarks, driven by inter-agent message passing and redundant context propagation.

It is therefore unclear whether the observed gains stem from the architectural choice itself or simply from the additional compute budget that multi-agent designs consume. This proposal investigates planner-based two-agent designs under *compute-matched* comparisons, to isolate the architectural contribution from the effect of the additional inference cost.

We hypothesise that planner-executor designs outperform compute-matched single-agent baselines on high-constraint queries, where explicit sub-goal decomposition aids constraint tracking, but offer no advantage on low-constraint queries where the planning step is trivial.

Our goal can be defined in two parts:

- **Goal 1:** To implement a minimal planner-plus-executor two-agent design and evaluate it on a planning-heavy benchmark, comparing it against a single-agent baseline.
- **Goal 2:** To compare the planner-based design against single-agent baselines given equivalent compute budgets, in order to determine whether the architectural gains persist when the baseline is given matched resources.

To achieve these goals, the following research questions (RQ) form the basis of our research:

- **RQ1:** What accuracy improvement does a planner-plus-executor design provide over a single-agent baseline on the TravelPlanner benchmark?
- **RQ2:** Does the planner-executor improvement persist when single-agent baselines are given a matched token budget,

via (a) self-consistency and (b) a single-agent plan-then-execute prompt?

- **RQ3:** Under which task conditions (e.g., constraint density, trip length, budget tightness) does the planner-based design justify its additional runtime and token cost?

By the end of this research, we expect to contribute in two ways. First, a reproducible implementation of a planner-executor system evaluated on TravelPlanner dataset. Secondly, a compute-matched analysis that clarifies when, if at all, the planner-based architecture provides value beyond the additional inference cost it consumes.

2 Related Work

Single-agent prompting techniques form the baseline for most LLM-based reasoning work. COT prompting [5] showed that intermediate reasoning steps substantially improve accuracy on arithmetic and commonsense tasks. ReAct [8] extended this by interleaving reasoning with tool use, forming the dominant single-agent paradigm for agentic benchmarks. Self-consistency [4] further improves single-agent accuracy by sampling multiple reasoning paths and aggregating their answers, at the cost of additional compute. Even though all of these single-agent techniques have improved the reasoning and responses of the models it still have a far way to go on some tasks. For example, extensive planning.

TravelPlanner [7] provides a constraint-heavy planning benchmark on which current LLM agents perform poorly only 0.6% GPT-4-Turbo (OpenAI, 2023) [7]. This dataset is a constraint-heavy planning benchmark in which an LLM agent must produce a multi-day itinerary which includes flights, accommodation, restaurants, and attractions. These objects have to be picked so that they satisfy both implicit commonsense rules and explicit hard constraints (budget, cuisine, room type, transport mode) drawn from a natural-language user query. Making it suitable for studying planning-focused architectural choices.

Within single-agent prompting, Plan-and-Solve separates planning from execution inside one model call, and adds a verbal self-critique loop [2]. Frameworks such as AutoGen [6] generalise these patterns into configurable multi-agent conversations. These approaches consistently report gains over single-agent baselines, though the compute cost of the baseline is barely measured to compare with single agent baselines. Recent critical work has questioned whether multi-agent gains persist under fair compute comparisons. [3] Tran and Kiela establish via both theory (Data Processing Inequality) and experiment that single-agent systems match or outperform multi-agent systems on multi-hop reasoning under matched thinking-token budgets. This is motivating the matched-compute framing of this proposal to check whether architecture matters in planning, or if it is about the token amount.

3 Methodology

This section details the steps taken to answer each research question. First, we implement the planner-executor system and a single-agent baseline. Next, we define matched-compute baselines. Finally, we analyze where the architectural design pays off.

3.1 On answering RQ1

We implement a planner-plus-executor two-agent design on top of the TravelPlanner benchmark [7]. The planner agent receives the query and produces a structured plan enumerating the sub-goals (e.g., flight search, accommodation, daily itinerary, budget allocation). The executor agent consumes the plan step by step, issues tool calls against the TravelPlanner sandbox, and maintains running state such as remaining budget and visited items. Both agents use the same base model to isolate the architectural effect from model mixing. As a baseline we implement a single-agent ReAct loop [8] with access to the same tool set. We report the TravelPlanner micro and macro pass rates on the validation split. All experiments use GPT-4o-mini as the base model, chosen to avoid both floor effects and the token-accounting ambiguity introduced by reasoning models with hidden thinking traces.

3.2 On answering RQ2

To determine whether the planner-executor gain persists under matched compute, we evaluate two single-agent baselines, both calibrated to the planner-executor system’s mean token usage per query: Self-consistency. Following Wang et al [4], we sample N independent ReAct trajectories and select the final plan via rubric-based verification: the model scores each candidate against TravelPlanner’s commonsense and hard-constraint checks, and the highest-scoring plan is returned. N is chosen such that mean total tokens match the planner-executor system.

Plan-then-execute (single agent) A two-call sequential pipeline using the same model and system prompt: the first call produces a structured plan; the second consumes the plan and produces the final itinerary. This isolates the agent role split from the planning step itself. If this baseline matches the two-agent system, the architectural boundary contributes nothing beyond the planning prompt.

Compute matching. We define matched compute as total input-plus-output tokens per query, averaged across the evaluation set, including all inter-agent messages. This penalises the planner-executor system for context-passing overhead, which we view as appropriate, because production deployments pay this cost. Tran and Kiela [3] use a narrower definition (thinking tokens only, excluding prompt and final output); we additionally report results under their definition for comparability. All configurations use GPT-4o-mini and are run with at least three seeds. We report mean accuracy with 95% paired-bootstrap confidence intervals over queries (resampled within seed, then aggregated across seeds), alongside seed standard deviation. We additionally report mean tokens per query for every configuration to verify the match holds within 10%.

3.3 On answering RQ3

Given the results from RQ1 and RQ2, we analyze when the planner-based design is worth its cost. We stratify TravelPlanner queries

by trip length, number of hard constraints, and budget tightness, and compute accuracy-per-thousand-tokens for each configuration within each stratum. This yields a profile of the task conditions under which the architectural choice justifies its additional runtime and token cost.

4 Planning

The study is split along the three research questions. The planning below is a guideline; the hard deadlines are:

- Final proposal submission: [May 3rd]
- Draft paper submission: [June 21]
- Final paper submission: [June 28]
- Conference presentation: [3 July]

Table 1: Planning Table

Task	Month 1				Month 2				Month 3			
	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Proposal												
Literature												
RQ1 impl.												
RQ2 impl.												
RQ3 analysis												
Draft paper												
Final paper												
Presentation												

References

- [1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 11733–11763.
- [2] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [3] Dat Tran and Douwe Kiela. 2026. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460 [cs.CL] <https://arxiv.org/abs/2604.02460>
- [4] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*.
- [5] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, Vol. 35. Curran Associates, 24824–24837.
- [6] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*.
- [7] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. TravelPlanner: A Benchmark for Real-World Planning with Language Agents. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR.

- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
- [9] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. 2025. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=LkzuPorQ5L>